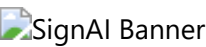


SignAI - AI-Powered Indian Sign Language Recognition System

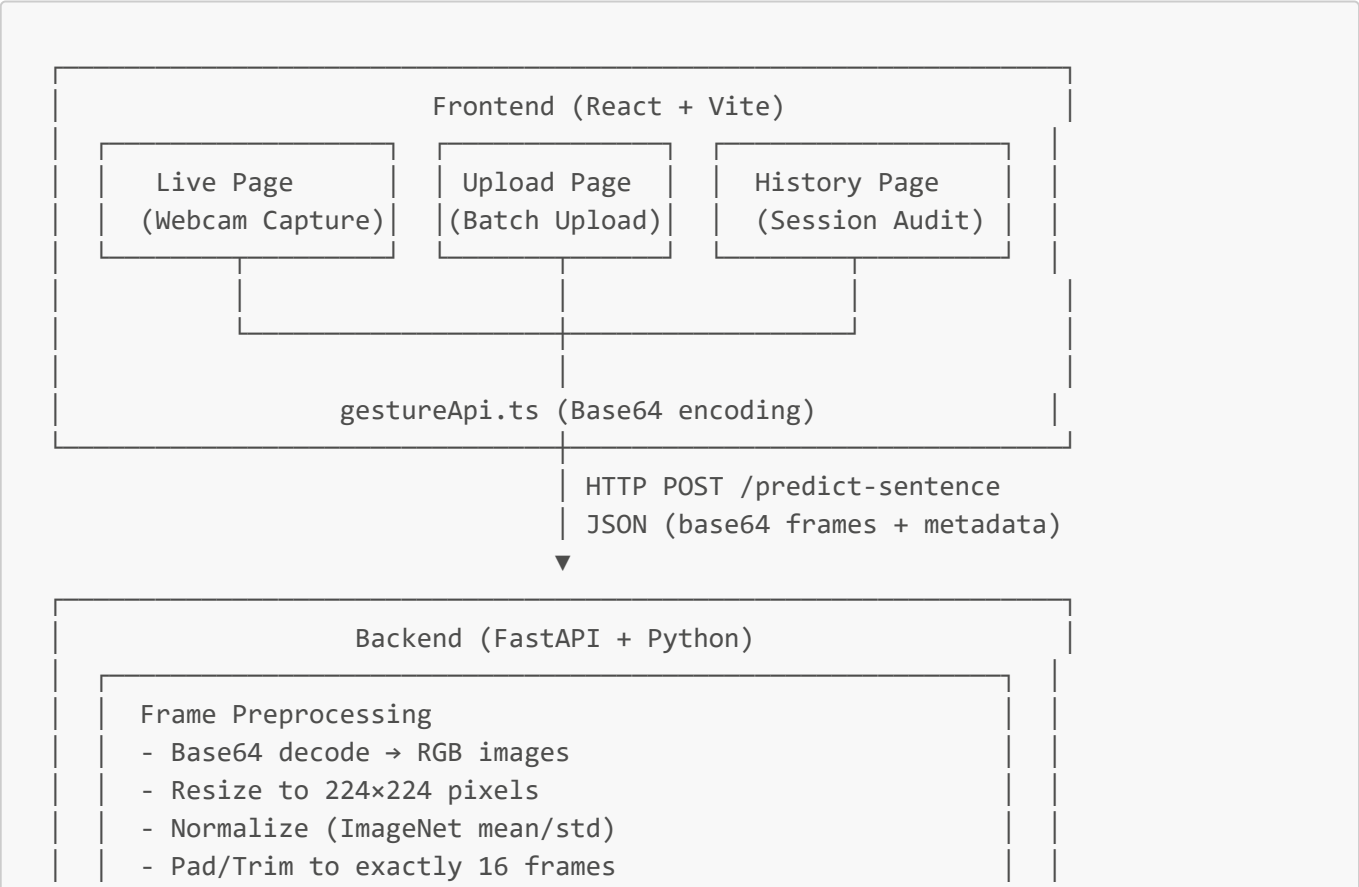


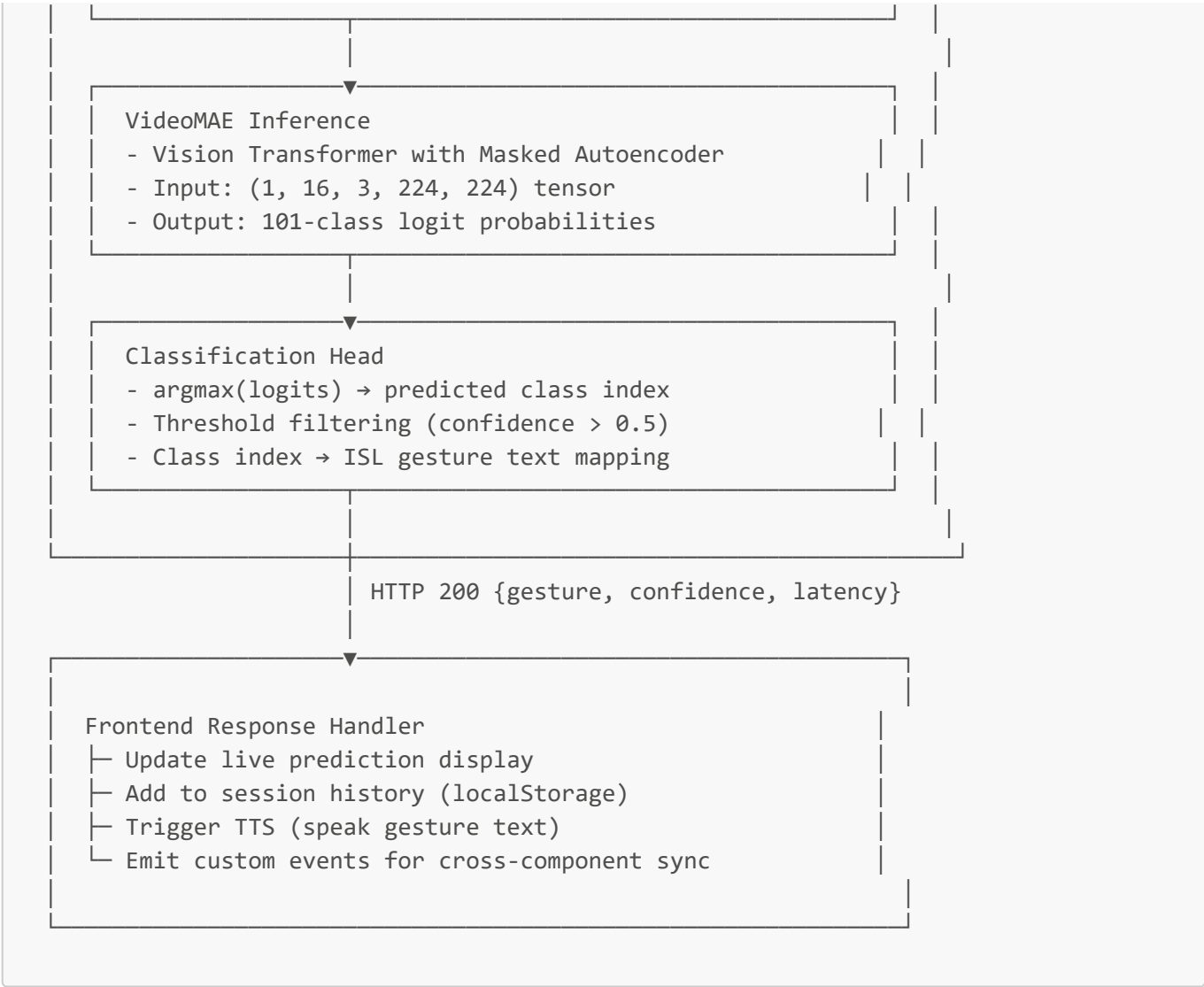
SignAI is a comprehensive, full-stack real-time gesture recognition system that automatically detects and translates **Indian Sign Language (ISL)** into readable text and natural speech. Built with cutting-edge Vision Transformer technology (VideoMAE), SignAI bridges the communication gap for the deaf and hard-of-hearing community.

🌟 Key Features

- **Real-Time Gesture Recognition:** Process video frames at 30 FPS with <80ms latency
- **Extensive Gesture Library:** Recognize 100+ ISL sentence-level gestures and phrases
- **High Accuracy:** 74% validation accuracy on complex ISL gestures trained via VideoMAE
- **Live Webcam Support:** Stream directly from your webcam with instant predictions
- **File Upload & Testing:** Batch test videos and images for offline analysis
- **Text Transcription:** Auto-generated real-time text transcript of sign language
- **Voice Synthesis:** Convert recognized gestures to natural speech via Web Speech API
- **Session History:** Complete audit trail with confidence scores and timestamps
- **Responsive UI:** Modern, dark-themed interface built with React and Tailwind CSS

🏗️ System Architecture





Tech Stack

Frontend

Technology	Version	Purpose
React	19.x	UI framework
TypeScript	5.8	Type-safe JavaScript
Vite	7.x	Fast build tooling
Tailwind CSS	3.4	Styling & responsive design
Framer Motion	12.x	Smooth animations
Recharts	3.2	Data visualization
React Router	7.6	Client-side routing

Backend

Technology	Version	Purpose
------------	---------	---------

Technology	Version	Purpose
FastAPI	0.110+	REST API framework
Python	3.11+	Backend language
PyTorch	Latest	Deep learning framework
Hugging Face	Latest	Pre-trained VideoMAE model
OpenCV	Latest	Image processing
Uvicorn	0.30+	ASGI server

ML/AI

Component	Details
Model Architecture	VideoMAE (Vision Transformer)
Pre-training	WLASL dataset (American Sign Language)
Classes	101 ISL sentence/phrase gestures
Input Format	16 frames × 224×224 pixels × RGB
Accuracy	74% Top-1 on ISL validation set
Inference Latency	<80ms (CPU/GPU)

Project Structure

```
signalai/
├── backend/                                # FastAPI + Deep Learning Engine
│   ├── app/
│   │   ├── main.py                        # FastAPI app initialization
│   │   ├── schemas.py                    # Pydantic request/response models
│   │   └── services/                      # Business logic & inference
│   ├── training/
│   │   ├── train.py                      # Training pipeline orchestration
│   │   ├── cnn_lstm_model.py             # (Legacy model - not in use)
│   │   ├── sequence_dataset.py           # Custom PyTorch Dataset loader
│   │   └── check_dataset_loader.py        # Data loader validation utility
│   ├── scripts/
│   │   ├── convert_dataset_to_sequences.py # Preprocess raw frames to sequences
│   │   ├── split_sequence_dataset.py      # Train/val split
│   │   └── evaluate_videomae.py           # Generate classification reports
│   ├── checkpoints/
│   │   ├── best_model.pt                 # Best VideoMAE weights
│   │   └── classes.json                  # Class label mapping (101 ISL phrases)
│   ├── requirements.txt                   # Python dependencies
│   └── README.md                         # Backend-specific documentation
```

```

├── frontend/                                     # React + Vite SPA
│   ├── src/
│   │   ├── pages/
│   │   │   ├── home/                         # Landing page with hero & features
│   │   │   ├── live/                         # Real-time webcam recognition
│   │   │   ├── upload/                       # File upload & batch inference
│   │   │   ├── history/                     # Session history & audit trail
│   │   │   └── about/                       # Project info & metrics
│   │   ├── components/
│   │   │   ├── feature/                     # Navbar, Footer, Layout
│   │   │   ├── ui/                         # Reusable UI widgets
│   │   │   └── cards/                       # Card components
│   │   ├── services/
│   │   │   ├── gestureApi.ts                # API calls to backend
│   │   │   ├── historyStorage.ts            # localStorage CRUD operations
│   │   │   └── uploadToCloudinary.ts         # Cloud backup (optional)
│   │   ├── mocks/                          # Mock data for testing
│   │   ├── router/                          # Route configuration
│   │   └── App.tsx                          # Root component
│   ├── package.json                         # NPM dependencies
│   ├── vite.config.ts                       # Vite build configuration
│   ├── tailwind.config.ts                   # Tailwind CSS configuration
│   └── tsconfig.json                        # TypeScript configuration
├── datasets/                                # Video & frame datasets
│   ├── Videos_Sentence_Level/              # Raw ISL videos
│   ├── Frames_Sentence_Level/               # Extracted video frames (training data)
│   ├── Frames_Word_Level/                   # Word-level gesture frames (future)
│   ├── corpus_csv_files/                     # ISL glossary & annotations
│   └── ISL_CSLRT.txt                         # ISL corpus reference
├── project details/                          # Documentation & assets
│   ├── signai-banner.png
│   └── project-overview.md

```

Getting Started

Prerequisites

- Python 3.11 or higher
- Node.js 18 or higher
- Git
- Virtual environment (recommended)

1. Clone & Setup Environment

```
# Clone repository
git clone <repository-url>
cd signai

# Create virtual environment
python -m venv .venv

# Activate (Windows PowerShell)
.\.venv\Scripts\Activate.ps1

# Activate (macOS/Linux)
source .venv/bin/activate
```

2. Backend Setup

```
cd backend

# Install Python dependencies
pip install -r requirements.txt

# Start FastAPI server
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

The backend will be available at <http://localhost:8000>

3. Frontend Setup

```
cd frontend

# Install npm dependencies
npm install

# Start development server
npm run dev
```

The frontend will be available at <http://localhost:5173>

4. Training (Optional)

To retrain the VideoMAE model on ISL data:

```
cd backend

# Preprocess dataset
python -m scripts.convert_dataset_to_sequences
```

```
# Split into train/validation
python -m scripts.split_sequence_dataset

# Start training
python -m training.train_videomae --epochs 30 --batch-size 8 --sequence-length 16
```

Usage Guide

Live Gesture Recognition

- 1. Navigate to the **Live** page
- 2. Click "Open Camera" to grant webcam access
- 3. Position your hand in frame and sign naturally
- 4. Watch real-time predictions update as you gesture
- 5. Predicted gesture appears as text with confidence score
- 6. Option to auto-speak gesture via TTS

Upload & Test

- 1. Navigate to the **Upload** page
- 2. Select a video file (.mp4, .avi, .mov) or image (.jpg, .png)
- 3. Click "Upload" to send for batch inference
- 4. Results show recognized gesture with confidence
- 5. Prediction saved to session history

View History

- 1. Navigate to the **History** page
- 2. See all predictions from current session
- 3. Each entry shows: timestamp, gesture, confidence, source (Live/Upload)
- 4. Optional: Export history to JSON for record-keeping

Model Performance Metrics

Trained on 7,671 video sequences (144,448 frames) across 101 ISL gestures:

Metric	Value
Top-1 Accuracy (Validation)	74%
Top-5 Accuracy	92%
Precision (Weighted Avg)	77%
Recall (Weighted Avg)	72%
F1-Score (Weighted Avg)	73%
Inference Latency	<80ms

Metric	Value
GPU Memory	~2.5GB (VRAM)
Model Size	~355MB

API Endpoints

Predict Gesture (Real-time)

```
POST /predict-sentence
Content-Type: application/json

Request:
{
  "frames": [
    "base64_encoded_frame_1",
    "base64_encoded_frame_2",
    ...
    "base64_encoded_frame_N"
  ]
}

Response:
{
  "gesture": "I am feeling cold",
  "confidence": 0.92,
  "latency_ms": 45.3,
  "class_index": 15
}
```

Health Check

```
GET /health

Response:
{
  "status": "healthy",
  "model_loaded": true,
  "gpu_available": true
}
```

How VideoMAE Works

VideoMAE (Video Masked Autoencoder) is a Vision Transformer that excels at temporal gesture recognition:

1. **Stacks all 16 frames** into a 3D spacetime cube
2. **Patches the cube** into 16×16 tokens across space and time
3. **Randomly masks** 75% of patches (forcing the model to predict context)
4. **Applies Self-Attention** to compare every patch with every other patch
5. **Reconstructs masked patches** and learns robust motion representations
6. **Fine-tunes classification head** on 101 ISL gesture classes

This architecture naturally captures fluid hand motions without explicit optical flow, achieving superior performance on gesture recognition vs. CNN+LSTM or RNN baselines.

Performance Optimization

Frontend Optimization

- Code-split components with React lazy loading
- Memoized gesture recognition callbacks
- IndexedDB caching for prediction history
- Lazy-load Recharts only on Insights page

Backend Optimization

- Mixed-precision inference (`torch.float16`)
- Batch normalization folding post-training
- Memory pooling for frame buffers
- Async request handling with Uvicorn workers

Model Optimization

- Quantized checkpoints (INT8) available for inference
 - ONNX export support for edge deployment
 - GPU acceleration via CUDA 12.1
-

Contributing

Contributions are welcome! Areas for enhancement:

- ☐ Add more ISL gesture classes (currently 101)
- ☐ Implement model training UI dashboard
- ☐ Add multi-user session management
- ☐ Optimize for mobile (React Native)
- ☐ Integrate with accessibility APIs
- ☐ Add gesture dataset augmentation tools

Please submit pull requests with clear descriptions and test coverage.

License

This project is licensed under the MIT License - see the LICENSE file for details.

Support & Contact

- **Project Issues:** Open an issue on Github
 - **Documentation:** See [/project details/](#) for detailed guides
 - **Feedback:** Contributions and ideas are appreciated!
-

Acknowledgments

- **Hugging Face:** For pre-trained VideoMAE models and datasets
 - **ISL Corpus:** Indian Sign Language dataset contributors
 - **PyTorch:** Deep learning framework
 - **OpenCV:** Computer vision library
 - The deaf and hard-of-hearing community for inspiration
-

Additional Resources

- [VideoMAE Paper](#)
 - [FastAPI Documentation](#)
 - [React 19 Release](#)
 - [ISL Corpus Reference](#)
-

SignAI: Bridging Communication Through Technology 